

A Supreme Court Time Machine

Abstract:

The U.S. Supreme Court drifts on issues as the country and its membership change. It is quite challenging for those who haven't studied the institution to understand these changes over time or contextualize the current court in comparison with the past, let alone predict what the Court today would do differently from the past. Comparing different eras of the Court with each other is also challenging because it begs the hypothetical question "what would this particular group of people have done if they had seen an otherwise unseen case". In this project, I wish to use machine learning models to compare the time periods of the court in this way.

Introduction:

The attempt to predict the Supreme Court's (SCOTUS) rulings has captured many of the best and brightest legal and statistical minds. Not only is the Court the source of high stakes outcomes, but it also lacks the same transparency and accountability that the other branches of government do. The ability to have a good idea of what the Court is going to do, and especially how it has changed from the past, is vital for politicians and citizens alike to understand the direction of law in the U.S.

The law is a subject of language. This is not an area most people believe computers excel in, but through natural language processing strategies our digital friends can become powerful allies.

It's not a new idea to attempt to predict the Supreme Court with AI. However, I have yet to see comparisons among time periods of the Court using ML models as representations of the Court itself. Based on a preliminary version of what I eventually hope this project can be, the results are quite promising. The data is out there to train the models, and the models predict with reasonable accuracy in their era (especially given they are only told the facts of a case).

Methods:

In a way, the Court looking at the facts summary of a case to decide it resembles a model predicting that case's outcome. Imagine defining the outcome of the case as the first of the two legal parties winning. Then, given a set of facts and prior beliefs of the judges, decide the outcome as a true or false value. Well, that makes the facts X , the trained model is the prior beliefs, and the outcome is y . Now we have a machine learning problem.

I'm far from the first to propose this method. There's even a Kaggle competition called "Supreme Court Judgement Prediction" doing this very analysis which I used as a starting point for my project. What I want to do differently is to use this method as a tool to compare different eras. I want someone to be able to ask how the Court in the 50s, 80, or 2010s would have decided a case we see today.

To do this, the arguably most important step is to process the fact summary paragraphs. First many of the facts came in HTML form, and thus inside `<p>` tags. My first step was to remove those. Then I formatted the text by removing non-alphanumeric characters and changing it to all lowercase. I then removed all english stopwords to capture a higher percentage of semantic meaning. I then tokenized the text using spacy, and lemmatized the tokens into their most basic root form of the words.

I then needed to divide the data into eras of the Court. In this preliminary version of a final product I used only two time periods: one before the Rehnquist Court and one after (including the Rehnquist Court). This was a good place to draw the line for the eras of the Court because it's generally considered the beginning of the conservative direction we see the current court in now, so it's a line that captures a notable ideological shift. Hopefully, the results will show a difference in predictions between the two Court eras.

Finally, I trained separate models for each time period in question. These serve as my case outcome predictors. For choosing the models, I ran a large grid search cross validation over RandomForest, LogisticRegression, and KNN, along with their hyperparameters. Eventually, I will try more models, but this served the purpose for the preliminary version of the project. All performed relatively similarly and performance was sometimes dependent on how I split court eras, so I decided to just always choose the best performing model using `grid_search's best_estimator_value`. In the demo, I then took cases none of them had seen and asked for results to compare, contrast, and for a user to ask about specific case results.

Data:

I used two data sources for this project - Kaggle and the SCOTUS database. The kaggle data is pulled from the Kaggle competition "Supreme Court Judgement Prediction". The dataset includes case fact summaries scraped from the legal website Oyez along with details about the cases like date, docket, topic, ruling type, etc. For my purposes, I only needed the fact summaries, the date (for determining court era), and the self-explanatory "first_party_winner" column. The data includes cases from 1942 through 2021 and contains 3098 rows.

The SCOTUS data was more challenging. The Supreme Court database is very in-depth and has many features useful for plenty of analysis goals. It does not have a fact summary associated with each case. Obviously this is a problem. Initially I had devised a web scraper for taking the facts summary straight from the HTML of each Oyez case (specified by term and docket number), but after getting this partially working I discovered an API-wrapper specifically for this use case. From the SCOTUS data, I pulled every docket number, got the facts from Oyez, and added the facts summary to the associated row of data. After saving this into a csv so as to not repeat that calculation, I filtered out all the rows where the facts were not present, and ended up with 3300 rows.

Next, merging the data was necessary. First, I created new standardized docket value columns, where all dockets were in the format yy-d where (y) is a year digit and (d) is a docket

number between 1 and 9999. These are the merge keys. I then completed a full outer join of the SCOTUS data and the kaggle data such that all cases present in either dataset or both were kept. A problem became self evident here. The merged data only had 3322 rows. This means that the kaggle data had almost all the same set of cases and facts as the entirety of Oyez. I'll discuss this more in the future plans section, but this wasn't what I had hoped.

Results:

The results seemed to be of mixed quality - which is very much what I expected for this version of the project. Firstly, the accuracies of the models were in the mid-60s. This makes a lot of sense. In the best of the Kaggle competition results, the model was achieving close to 80%, but that was on close to twice as much data and incorporating features other than the facts summaries. My goal in this version of the project was never to maximize accuracy. I cared much more about creating a mimic of the Court in my models, not a black box machine which is "more accurate". I wanted to have models which looked at nothing but the substantive information that is given to the Justices themselves and "decided" cases.

In terms of the similarity of models to themselves, this was very much in the range I expected. As one can see in the demo, the pre-rehnquist and post-rehnquist models predicted the same outcome approximately 75% of the time. I would be quite worried if this value was much closer to 0 or 100. Closer to 0 would imply the eras of the Court decide cases wildly differently, which is not true. Most SCOTUS cases are unanimous or close to it, and time rarely changes how the institution looks at those cases. Closer to 100 would imply there is almost no difference between the eras, which would also be surprising because though *most* cases are clear, many (especially those which Oyez has entries for) are contentious and have changed with time.

The user-input works as expected, but real results are difficult to glean from manually pasting notable cases and checking output - especially because the Court often pretends never to rule a past case as incorrect (they instead change legal precedent through taking new *similar* cases and ruling differently). The tool functions, and that's what I care about more for now.

Output and figures for results below:

Model scores on pre-2021 case data:

```
Pre-rehnquist-model score: 0.6617357656163626
Post-rehnquist-inclusive-model score: 0.6634747983614482
```

Example of successful test output:

```
-----STARTING TESTS-----
Tests passed
-----ENDING TESTS-----
```

Example of demo output and a user asking for one case outcome prediction:

```
The pre-rehnquist-court and post-rehnquist-court (inclusive) predicted the same case outcome 75.63%
Type "exit" to quit at any step:
Enter a year: 1973
Enter a facts summary: In 2018, Mississippi passed a law called the "Gestational Age Act," which prohib
ckson Women's Health Organization, the only licensed abortion facility in Mississippi, and one of its d
requesting an emergency temporary restraining order (TRO). After a hearing, the district court granted th
trict court granted the clinic's motion for summary judgment and enjoined Mississippi from enforcing th
be viable at 15 weeks, and Supreme Court precedent prohibits states from banning abortions prior to via
[1.]
First party wins
```

Discussion:

First, I'd like to discuss challenges I ran into during this project. The single biggest obstacle I encountered - and what took most of my time - was dealing with data. First, the scraping of fact summaries to incorporate with the SCOTUS data was not trivial. I ran into issues merging the kaggle and SCOTUS data as well, though those were all resolved. Unfortunately, the lack of distinct data between those two sources was not a solvable problem without sourcing facts from outside Oyez, which felt out of the scope of this preliminary product.

Now for future plans for this project. Firstly, I think it's important in the next version to consider the issue/topic of the case both in the methodology and analysis of the results. Due to inconsistency in the issue definitions between kaggle and the SCOTUS data, I declined to add issue type into the features for models. I felt it was more important for a preliminary product to connect with the extra SCOTUS data than to limit the current and future versions to only use the kaggle dataset. Also, due to the "first_party_winner" format of results, incorporating issue type is even more challenging. For example, in a case with the issue of "free speech", we don't have results that say "less free speech result" or "more free speech result". The first party in a Supreme Court case is just the one that lost at the Appellate level. This information is almost always captured in fact summaries, but not in issue type. All that said, I believe if it can be successfully implemented the topic/issue of the case will be an extremely useful feature for predictive accuracy, especially as it compares different time periods of the Court.

Next, again we have to return to the data size issue. Oyez is not the only site that contains fact summaries of cases. It won't be simple, but I plan to look for other sources and hopefully increase the data size by more than 3 times. In addition to increasing accuracy and allowing more eras of the court (every 10 years or every Chief Justice), it would also allow a look further back in history than 1941. Of those improvements, the most important one is allowing more eras. If we could track the Court by decade, I think the results would be easier to interpret and would align more closely with what academics already understand about the courts.

Lastly, I want to make the user interface better. Especially after incorporating issue type and more eras, this tool would be something I'd actually be interested in using - if for nothing else other than the novelty of it. I plan to attend law school, and evolving this project over time into an app with many pre-trained models could be a really interesting topic to investigate as I learn more about the law itself.

References:

- Data
 - <https://www.kaggle.com/datasets/deepcontractor/supreme-court-judgment-prediction/data>
 - <http://scdb.wustl.edu/data.php>
- Sources
 - Haribalanc. “Supreme Court Judgement Prediction.” Kaggle, Kaggle, 17 May 2023, www.kaggle.com/code/haribalanc/supreme-court-judgment-prediction/notebook.
 - Raghavkachroo. “Supreme Court Judgement Prediction.” Kaggle, Kaggle, 9 Feb. 2022, www.kaggle.com/code/raghavkachroo/supreme-court-judgment-prediction.
 - Various in-class assignments for Northeastern’s CS4120 Course with Professor Tahmasebi
 - Library documentation for pandas, numpy, nltk, regex, spacy, sklearn, joblib, oyez-api-wrapper

Appendix:

- Instructions on running the code are present in the README
- Code Structure
 - SupremeCourtTimeMachine.ipynb
 - External Dependencies:
 - pandas, numpy, nltk, regex, spacy, sklearn, joblib
 - Classes/Functions/Methods:
 - standardize_term: str->int
 - standardizes all ‘term’ values to a 4-digit year representation
 - determine_era: int->str
 - given a year, determines a named court-era
 - get_column_value_by_era: str, str->series
 - given an era and column name, gets the series corresponding to that column’s values for the era
 - train_test_split_for_era: str->numpy array, numpy array, numpy array, numpy array
 - given an era, returns the X_train, X_test, y_train, y_test for that era’s cases for use in model training
 - fit_and_score_model_for_era: str->sklearn model, float

- given a court era, trains a model on data from that era and scores the model on accuracy
- Input/Output:
 - Input:
 - csv files justice, SCDB_2023_01_caseCentered_Docket, SCOTUS_data_with_Facts
 - Output:
 - csv files data_by_era, data_by_era_pre_2021, fully_processed_data, fully_processed_data_by_era, fully_processed_data_pre_2021, fully_processed_data_post_2021
 - sklearn models pre-rehnquist_model, post_rehnquist_inc_model
- oyez_script.py
 - External Dependencies:
 - pandas, oyez_api_wrapper
 - Classes/Functions/Methods:
 - get_case_facts: pandas df row->None or str
 - fetch case facts from oyez api given a term and a docket from the row of the dataframe
 - Input/Output:
 - Input
 - csv file SCDB_2023_01_caseCentered_Docket
 - Output
 - csv file SCOTUS_data_with_facts
- demo.py
 - External Dependencies:
 - pandas, joblib
 - Classes/Functions/Methods:
 - make_predictions: pandas df, sklearn model->series
 - given data and a model, makes case outcome predictions for each row of data (assumes correctly formatted data)
 - compare_predictions: series, series->float
 - given two prediction series of the same length, determines their similarity percentage
 - predict_case_outcome: int, str-> str
 - predicts the outcome of a single case given a year to determine era and a facts paragraph describing the case
 - main: None->None

- loads processed data and models, makes predictions with models, compares them, and prints the similarity score.
 - Creates the UI loop for users to enter year and fact strings to receive a prediction of the case result from an era
- Input/Output:
 - Input
 - sklearn models pre_rehnquist_model, post_rehnquist_inc_model
 - csv files fully_processed_data_post_2021_inc
 - Output
 - Terminal ui
- test.py
 - External Dependencies:
 - joblib, pandas, numpy
 - Classes/Functions/Methods:
 - *these should be self explanatory, they test the function or file they describe
 - *they all have a None->None signature
 - test_standardize_term
 - test_determine_era
 - test_SupremeCourtTimeMachine
 - test_make_predictions
 - test_compare_predictions
 - test_demo
 - test_get_case_facts
 - test_oyez_script
 - Input/Output:
 - Input
 - csv files fully_processed_data_post_2021_inc
 - sklearn models post_rehnquist_inc_model
 - Output
 - terminal string results